

Informe Final — Desafío I

Implementación Sweet Crush mediante manipulación de bits en C++

PAULINA GÓMEZ GARCÍA

C.C 1033178089

Ingeniería en Telecomunicaciones

PABLO ANDRES CARRILLO YEPES

C.C 1118824441

Ingeniería en Telecomunicaciones

Introducción

El desafío consiste en implementar el juego *Sweet Crush*, un juego de emparejamiento donde el jugador elimina fichas que forman secuencias de tres o más iguales en horizontal o vertical, generando reorganizaciones (cascadas) del tablero. El sistema debe soportar como mínimo seis tipos de fichas distintas, detectar automáticamente los emparejamientos y procesar las cascadas resultantes hasta que el tablero se estabilice.

Sin embargo, el objetivo real de la actividad no es el juego en sí, sino evaluar el dominio y lo aprendido en clase de conceptos fundamentales de C++: manipulación de bits, gestión de memoria dinámica y programación estructurada. La particularidad que distingue este desafío de una implementación convencional de un juego tipo match-3 es la restricción de representar cada ficha del tablero en exactamente 3 bits, sin desperdiciar espacio ni alinear a byte, lo que obliga a trabajar directamente con operadores bitwise (&, |, ^, ~, <<, >>) en cada acceso o modificación del tablero, en lugar de usar estructuras de datos convencionales.

A. Análisis del problema

A partir del documento del desafío se identificaron como restricciones determinantes para el diseño de la solución: la prohibición de `class`, `struct`, `template`, `std::string` y contenedores de la STL; el uso exclusivo de tipos primitivos y punteros/arreglos dinámicos (`unsigned char*`); la representación compacta de cada ficha en exactamente 3 bits, empaquetados de forma continua sin relleno, incluso cuando una ficha queda repartida entre dos bytes; la obligatoriedad de manipular esos bits mediante operadores bitwise, sin desempaquetar el tablero a una matriz auxiliar; la gestión dinámica de memoria exacta al tamaño lógico del tablero, sin sobredimensionar desde el inicio; y la regla del 65%, según la cual la memoria física solo se reduce al eliminar filas o columnas cuando el uso cae por debajo de ese umbral.

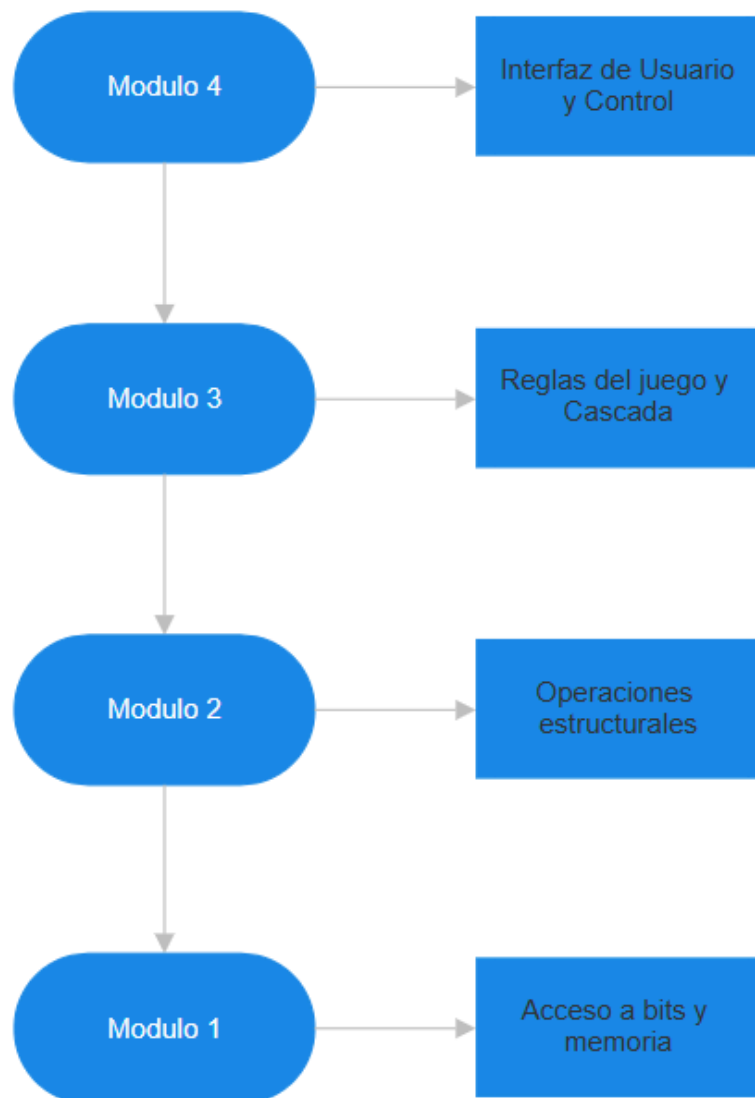
Estas restricciones tuvieron un impacto directo sobre las decisiones de diseño. La prohibición de POO y STL obligó a recargar las firmas de las funciones con múltiples parámetros por referencia, ya que, al no existir un objeto que agrupara los datos, el estado del programa debía vivir en el controlador principal y ser arrastrado manualmente a cada módulo. La representación compacta de 3 bits por ficha, junto con la manipulación bitwise obligatoria, hizo necesario centralizar esa lógica en un par de funciones primitivas de lectura y escritura: manipular los desplazamientos de bits repartidos entre bytes directamente dentro de la lógica del juego habría vuelto el código difícil de mantener y depurar. Por su parte, la gestión dinámica exacta junto con la regla del 65% obligó a separar la dimensión lógica del tablero de la dimensión física reservada en memoria, llevando ambos valores en paralelo para evitar pedir y liberar memoria del heap de forma innecesaria en cada operación.

Estas restricciones no buscan únicamente dificultar la implementación, sino evaluar destrezas que suelen quedar ocultas cuando se usan librerías de alto nivel: si el estudiante realmente comprende cómo se organiza la memoria detrás de esas abstracciones, si ejerce el razonamiento a nivel de bits en lugar de simular cumplirlo sobre una copia expandida del tablero, y si maneja correctamente `new/delete` en vez de evitar los redimensionamientos reales reservando de más desde el principio.

Este análisis se mantuvo vigente durante la implementación: las restricciones identificadas en la fase preliminar resultaron ser, en efecto, las que más moldearon las decisiones de diseño a lo largo de los cuatro módulos, sin que surgieran restricciones adicionales no anticipadas.

B. Esquema de tareas definidas en el desarrollo de los algoritmos

La solución final se organizó en cuatro módulos, con la misma jerarquía de dependencia unidireccional planteada en el diseño preliminar, ya confirmada en el código real:



Módulo 1 — capa de infraestructura: *calcularlosbytesnece* (tamaño mínimo en bytes para $F \times C$ fichas), *creaciontablero/liberatablero* (reserva y liberación del bloque dinámico), *Obtener_Fichas/Establecer_fichas* (lectura y escritura de los 3 bits de una posición, incluyendo el caso de ficha repartida entre dos bytes), y Resultado_*PUNTERO_BLOQUE_MEMORIA_BINARIO* como utilidad de depuración.

Módulo 2 — manipula la estructura y el contenido del tablero, apoyándose únicamente en las funciones de Módulo 1: *Agregar_fila/Eliminar_fila* y *Agregar_Columna/Eliminar_columnas* (modifican las dimensiones en cualquier posición, no solo en los extremos, reservando en cada caso exactamente la memoria necesaria y aplicando la regla del 65% al eliminar); *Hacer_vivo_el_tablero* (gravedad: compacta las fichas no vacías hacia el fondo del tablero); *Llenar_espacios_vacios* (rellena con fichas aleatorias, distribución uniforme, toda posición vacía — se usa tanto para el relleno inicial del tablero como para el relleno posterior a una cascada); y *Imprimir_Tablero_Legible* como utilidad de depuración.

Módulo 3 — concentra la lógica del juego y el estado de la partida (puntuación, eliminaciones del usuario, fichas eliminadas, combinaciones detectadas, cascadas de la jugada actual): *detectarCombis* (recorre el tablero en dos pasadas, horizontal y vertical, marcando las posiciones que forman parte de una secuencia de 3 o más fichas iguales); *revisarCombis* (elimina las fichas marcadas y calcula la puntuación); *ejecutarCascada* (orquestra el ciclo detectar -> procesar -> gravedad -> relleno -> volver a detectar, hasta que el tablero se estabiliza).

Módulo 4 — interactúa con el jugador por consola: *dimensionesIni* (captura filas/columnas al iniciar), *dibujarFichas/dibujarBinario* (visualización en símbolos y en binario empaquetado real), *mostrarProgreso* (estado de la partida), *menuPrincipal/seleccionUsuario/ seleccionIndice* (captura de las jugadas y operaciones estructurales que pide el usuario).

El controlador principal (*main.cpp*) es quien posee las variables de estado y las dimensiones del tablero, y las distribuye por referencia a cada módulo según lo que cada uno necesita — un módulo nunca recibe más contexto del que su tarea requiere: Módulo 2, por ejemplo, nunca conoce la puntuación ni el número de cascadas, porque su responsabilidad se limita a la estructura y el contenido del tablero, no a las reglas del juego.

C. Algoritmos implementados

Módulo 1. Las dos funciones primitivas (*Obtener_Fichas* y *Establecer_fichas*) calculan, a partir de (fila, columna), el índice lógico de la ficha ($\text{fila} \times \text{columnas} + \text{columna}$) y su posición inicial en bits ($\text{índice} \times 3$), de donde se derivan el byte donde comienza y el desplazamiento dentro de ese byte. Si la ficha cabe dentro de un solo byte se extrae o se escribe con una máscara y un desplazamiento; si queda repartida entre dos bytes, se separa en su parte menos significativa (bits que caben en el primer

byte) y su parte más significativa (bits que continúan en el byte siguiente), combinándolas con operadores `&`, `|` y desplazamientos.

Módulo 2. Agregar_fila/Eliminar_fila reconstruyen el tablero completo en un nuevo bloque de memoria del tamaño exacto necesario, recorriendo las filas viejas y calculando su nueva posición con la expresión $\text{fila_vieja} + (\text{fila_vieja} \geq \text{posición})$ (o su resta equivalente al eliminar), que desplaza automáticamente las filas posteriores al punto de inserción o eliminación sin necesidad de una condición explícita. Agregar_Columna/Eliminar_columnas siguen la misma estrategia, pero aplicada dentro de cada fila, ya que insertar o eliminar una columna cambia el *stride* de todas las filas del tablero. *Hacer_vivo_el_tablero* implementa la gravedad recorriendo cada columna de abajo hacia arriba, con un puntero de escritura que también avanza de abajo hacia arriba y solo retrocede cuando encuentra una ficha no vacía — así una ficha nunca se lee después de haber sido movida. *Llenar_espacios_vacios* recorre el tablero y sustituye toda posición en ESTADO_VACIO por un valor aleatorio uniforme entre las 6 fichas válidas.

Módulo 3. La detección de combinaciones recorre el tablero en dos pasadas independientes (horizontal y vertical), marcando en un arreglo auxiliar las posiciones que forman parte de una secuencia de 3 o más fichas iguales. El principal reto de diseño fue garantizar que una ficha que participa simultáneamente en una combinación horizontal y una vertical se contará como parte de dos combinaciones distintas, pero se eliminará una sola vez — esto se logró marcando cada posición con un simple indicador binario en vez de contar eliminaciones por posición. El ciclo de cascada (detectar -> procesar -> aplicar gravedad -> rellenar) se repite hasta que una vuelta no encuentra combinaciones nuevas, con un tope de seguridad para evitar bloqueos ante posibles errores en los módulos de los que depende.

Módulo 4. El mayor reto de diseño no fue algorítmico sino de representación: traducir los 6 códigos de 3 bits a símbolos legibles en consola (P, G, &, !, X, Y,) sin depender de estructuras de datos prohibidas por el enunciado. La lectura de datos del usuario se centralizó en una única función de lectura validada, reutilizada por todas las funciones de captura, para garantizar el mismo comportamiento ante entradas inválidas en cualquier punto del programa.

D. Problemas de desarrollo que afrontó

Durante el desarrollo se identificaron y corrigieron varios errores mediante un proceso combinado de pruebas de escritorio a mano y pruebas automatizadas que comparaban el valor esperado contra el resultado real de cada función, lo que permitió aislar progresivamente la causa de cada falla.

En **Módulo 1** se identificó un error recurrente en la protección del header (`#ifndef/#endif` mal ubicado), que provocaba errores de redefinición al incluir el archivo desde múltiples módulos, y que persistió sin corregirse a través de varias entregas.

En **Módulo 2** se encontraron varios errores de igual gravedad: una variable de bucle reasignada sobre sí misma en `Agregar_Columna`, que corrompía los datos de la fila procesada; un puntero no indexado correctamente en `Eliminar_columnas`, que provocaba un cierre inesperado del programa (*segmentation fault*); y un recorrido en sentido contrario al de escritura en la función de gravedad, que causaba pérdida de fichas al reacomodar el tablero. Un primer intento de corrección de la gravedad introdujo, además, un nuevo error de condición en el bucle (fila $\geq$ filas en vez de fila ≥ 0), que dejaba la función sin efecto alguno — evidencia de que cada corrección debía verificarse con pruebas concretas y no darse por buena solo por inspección visual del código.

En **Módulo 3**, el error identificado fue que la función de detección de combinaciones dependía de que quien la llamara hubiera inicializado previamente el arreglo de marcas, lo cual se corrigió haciendo que la función se responsabilizará de su propia inicialización.

En **Módulo 4** se identificaron dos problemas de robustez en la lectura de datos por consola: la falta de validación ante texto no numérico, y un bucle infinito que se producía si la entrada de datos terminaba abruptamente (fin de archivo) mientras el programa esperaba un número — este último llegó a generar cientos de megabytes de salida repetida antes de ser detectado y corregido.

E. Evolución de la solución y consideraciones para tener en cuenta en la implementación

La integración de los cuatro módulos se construyó progresivamente mediante **contratos de función**: antes de tener las implementaciones reales de Módulo 1 y Módulo 2, se definieron sus firmas (nombres, parámetros y tipos) para que Módulo 3 y Módulo 4 pudieran compilarse y probarse de forma independiente contra versiones provisionales (*stubs*) de esas funciones. A medida que las implementaciones reales fueron entregadas, se sustituyeron los *stubs* sin modificar una sola línea de Módulo 3 o Módulo 4, lo cual confirmó en la práctica que el diseño por contrato cumplió su propósito: aislar el trabajo de cada integrante y detectar discrepancias de nombres o de comportamiento en el momento de compilar, no al final del proyecto.

El diseño de **Módulo 3** se mantuvo prácticamente igual desde su concepción inicial; los cambios se limitaron a ajustar los nombres de las funciones llamadas de Módulo 2

a medida que se confirmaban con la implementación real. **Módulo 4** tuvo cambios menores sobre el mismo diseño base: se acortaron los nombres de varias funciones y se agregó la captura de dimensiones iniciales, que no había sido contemplada explícitamente en el diseño preliminar. **Módulo 1** y **Módulo 2** evolucionaron de un primer borrador con errores de compilación y de lógica hacia una versión corregida y verificada, mediante pruebas automatizadas construidas específicamente para aislar cada función y comparar su resultado esperado contra el real antes y después de cada corrección.

Como consideración para futuras implementaciones similares: separar la integración en contratos (con mi compañera) de función antes de tener el código final de cada módulo permitió que el trabajo en equipo avanzara en paralelo sin bloquear a ninguna de las partes, y que los errores de cada módulo se detectaran y corrigieran de forma aislada, con evidencia concreta (una prueba que falla y luego pasa) en lugar de depender únicamente de la revisión visual del código.